

Tool implementation — models.py

src/lab_billing/forecast/models.py — git blob c6c9ee467b07 — v1.4.0 (2026-09-22)

```
"""Module B models (spec section 12).

B0: transparent age/stage empirical-risk baseline (rules, not probabilities).
B1: discrete-time multinomial logistic regression on interval rows.
C1: B1 plus reconstruction-quality features (candidate under test).

All models output three mutually exclusive conditional probabilities
(payment / close_no_payment / none) per interval, with cumulative
F_payment, F_close and S(h) = P(still open) computed by the product rule.
"""

from __future__ import annotations

import math
from dataclasses import dataclass, field
from typing import Any

import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

from .snapshot import Snapshot

FEATURE_COLS = [
    "age_days", "payer_commercial", "payer_government", "payer_self",
    "payer_unknown", "billed_amount_minor", "n_versions", "n_corrected",
    "n_voids", "n_payments_pre_t", "n_unknown_pre_t", "n_status_updates_pre_t",
    "n_closes_pre_t", "n_reopens_pre_t", "interval",
]
QUALITY_COLS = ["quality_penalty"]

OUTCOMES = ["payment", "close_no_payment", "none"]

@dataclass
class IntervalRow:
    snapshot_id: str
    lineage_id: str
    interval: int
    features: dict[str, Any]
    label: str # payment / close_no_payment / none

def build_interval_rows(snaps: list[Snapshot], horizon_days: int = 30,
                       interval_days: int = 5,
                       include_censored: bool = False) -> list[IntervalRow]:
    """Interval-level risk rows. Rows stop after the event; still-open rows
    continue through the last interval; censored snapshots produce no rows
    (v1)."""
    rows: list[IntervalRow] = []
    n_intervals = max(1, horizon_days // interval_days)
    for s in snaps:
        if s.outcome == "censored" and not include_censored:
```

```

        continue
    if s.outcome == "ambiguous":
        continue
    last = s.outcome_interval if s.outcome_interval is not None else n_intervals - 1
    for iv in range(last + 1):
        feats = _featurize(s, iv)
        if iv == s.outcome_interval:
            label = s.outcome
        else:
            label = "none"
        rows.append(IntervalRow(
            snapshot_id=s.snapshot_id, lineage_id=s.lineage_id,
            interval=iv, features=feats, label=label))
    return rows

def _featurize(s: Snapshot, interval: int) -> dict[str, float]:
    f = s.features
    out: dict[str, float] = {
        "age_days": float(f.get("age_days", 0.0)),
        "payer_commercial": 1.0 if f.get("payer_category") == "commercial" else 0.0,
        "payer_government": 1.0 if f.get("payer_category") == "government" else 0.0,
        "payer_self": 1.0 if f.get("payer_category") == "self" else 0.0,
        "payer_unknown": 1.0 if f.get("payer_category") == "unknown" else 0.0,
        "billed_amount_minor": float(int(f.get("billed_amount_minor", 0) or 0)),
        "n_versions": float(f.get("n_versions", 1)),
        "n_corrected": float(f.get("n_corrected", 0)),
        "n_voids": float(f.get("n_voids", 0)),
        "n_payments_pre_t": float(f.get("n_payments_pre_t", 0)),
        "n_unknown_pre_t": float(f.get("n_unknown_pre_t", 0)),
        "n_status_updates_pre_t": float(f.get("n_status_updates_pre_t", 0)),
        "n_closes_pre_t": float(f.get("n_closes_pre_t", 0)),
        "n_reopens_pre_t": float(f.get("n_reopens_pre_t", 0)),
        "interval": float(interval),
        "quality_penalty": float(f.get("quality_penalty", 0)),
        "payer_category_key": str(f.get("payer_category", "unknown")),
    }
    return out

class B0AgeStageBaseline:
    """Empirical risk by (payer category, age bucket). Not probabilities."""

    def __init__(self):
        self.table: dict[tuple, dict[str, float]] = {}

    def fit(self, rows: list[IntervalRow]) -> None:
        buckets: dict[tuple, dict[str, float]] = {}
        for r in rows:
            age = r.features["age_days"]
            bucket = (age < 15, 15 <= age < 45, age >= 45)
            key = (r.features["payer_category_key"], bucket)
            b = buckets.setdefault(key, {"payment": 0.0, "close_no_payment": 0.0, "none":
0.0, "n": 0.0})
            b["n"] += 1
            b[r.label] += 1
        for k, b in buckets.items():
            n = b["n"]

```

```

        self.table[k] = {
            "payment_rate": b["payment"] / n,
            "close_rate": b["close_no_payment"] / n,
            "support": int(n),
        }

def predict_interval(self, features: dict[str, Any]) -> dict[str, float] | None:
    age = features["age_days"]
    bucket = (age < 15, 15 <= age < 45, age >= 45)
    key = (features["payer_category_key"], bucket)
    entry = self.table.get(key)
    if entry is None or entry["support"] < 5:
        return None # insufficient support => ABSTAIN for this rule cell
    return {
        "payment": entry["payment_rate"],
        "close_no_payment": entry["close_rate"],
        "none": 1.0 - entry["payment_rate"] - entry["close_rate"],
    }

class DiscreteMultinomial:
    """B1 (and C1 with quality features): multinomial logistic regression
    on interval rows. Emits per-interval conditional probabilities."""

    def __init__(self, use_quality: bool = False):
        self.use_quality = use_quality
        self.model: LogisticRegression | None = None
        self.scaler: StandardScaler | None = None
        self.fitted = False
        self.min_class_support = 5

    @property
    def feature_cols(self) -> list[str]:
        cols = list(FEATURE_COLS)
        if self.use_quality:
            cols += QUALITY_COLS
        return cols

    def fit(self, rows: list[IntervalRow]) -> str:
        counts: dict[str, int] = {}
        for r in rows:
            counts[r.label] = counts.get(r.label, 0) + 1
        for cls in ("payment", "close_no_payment"):
            if counts.get(cls, 0) < self.min_class_support:
                return "INSUFFICIENT_DATA"
        X = np.array([[r.features.get(c, 0.0) for c in self.feature_cols] for r in rows])
        y = np.array([r.label for r in rows])
        self.scaler = StandardScaler()
        X = self.scaler.fit_transform(X)
        self.model = LogisticRegression(
            max_iter=5000, solver="lbfgs",
        )
        self.model.fit(X, y)
        self.fitted = True
        return "FITTED"

    def predict_interval(self, features: dict[str, float]) -> dict[str, float]:
        if not self.fitted:

```

```

        raise RuntimeError("model not fitted")
    X = np.array([[features.get(c, 0.0) for c in self.feature_cols]])
    X = self.scaler.transform(X)
    probs = self.model.predict_proba(X)[0]
    out = {cls: 0.0 for cls in OUTCOMES}
    for cls, p in zip(self.model.classes_, probs):
        out[cls] = float(p)
    return out

def cumulative_curves(interval_probs: list[dict[str, float]]) -> dict[str, list[float]]:
    """Product rule: S(0)=1; F_payment(h)=sum S(k-1)*p_payment(k); etc."""
    S = 1.0
    Fp, Fc = 0.0, 0.0
    f_payment, f_close, s_curve = [], [], []
    for p in interval_probs:
        if abs(sum(p.values()) - 1.0) > 1e-6:
            raise ValueError(f"interval probabilities do not sum to 1: {p}")
        if any(v < -1e-9 for v in p.values()):
            raise ValueError(f"negative interval probability: {p}")
        Fp += S * p["payment"]
        Fc += S * p["close_no_payment"]
        S = S * p["none"]
        f_payment.append(Fp)
        f_close.append(Fc)
        s_curve.append(S)
    return {"F_payment": f_payment, "F_close": f_close, "S": s_curve}

def evaluate(rows: list[IntervalRow], model) -> dict[str, float]:
    """Log loss and three-class Brier score on interval rows."""
    import numpy as _np

    losses, briers = [], []
    n = 0
    for r in rows:
        p = model.predict_interval(r.features)
        if p is None:
            continue
        target = r.label
        prob = max(p.get(target, 0.0), 1e-12)
        losses.append(-math.log(prob))
        brier = sum((p.get(c, 0.0) - (1.0 if c == target else 0.0)) ** 2
                    for c in OUTCOMES)
        briers.append(brier)
        n += 1
    if n == 0:
        return {"log_loss": float("nan"), "brier_three_class": float("nan"), "n": 0}
    return {"log_loss": float(_np.mean(losses)),
            "brier_three_class": float(_np.mean(briers)),
            "n": n}

def snapshot_probs(snap: Snapshot, predictor, horizon_days: int = 30,
                  interval_days: int = 5):
    """H-day final class probabilities for one snapshot, or None on abstain."""
    feats = _featurize(snap, 0)
    interval_probs = []

```

```

    for iv in range(max(1, horizon_days // interval_days)):
        feats["interval"] = float(iv)
        p = predictor.predict_interval(feats)
        if p is None:
            return None
        interval_probs.append(p)
    curves = cumulative_curves(interval_probs)
    return {
        "F_payment": curves["F_payment"][-1],
        "F_close": curves["F_close"][-1],
        "S": curves["S"][-1],
    }

def evaluate_snapshots(snaps: list[Snapshot], predictor,
                      horizon_days: int = 30, interval_days: int = 5) -> dict[str,
float]:
    """Snapshot-level Brier and log loss against the observed outcome."""
    briers, losses = [], []
    n_abstain = 0
    for s in snaps:
        if not s.label_known:
            continue
        probs = snapshot_probs(s, predictor, horizon_days, interval_days)
        if probs is None:
            n_abstain += 1
            continue
        target = {"payment": (1.0, 0.0, 0.0),
                  "close_no_payment": (0.0, 1.0, 0.0),
                  "still_open": (0.0, 0.0, 1.0)}[s.outcome]
        pvec = (probs["F_payment"], probs["F_close"], probs["S"])
        brier = sum((pvec[i] - target[i]) ** 2 for i in range(3))
        briers.append(brier)
        p_target = max(pvec[{"payment": 0, "close_no_payment": 1, "still_open": 2}
[s.outcome]], 1e-12)
        losses.append(-math.log(p_target))
    n = len(briers)
    if n == 0:
        return {"snap_brier": float("nan"), "snap_log_loss": float("nan"),
                "n_snapshots": 0, "n_abstain": n_abstain}
    return {"snap_brier": float(np.mean(briers)),
            "snap_log_loss": float(np.mean(losses)),
            "n_snapshots": n, "n_abstain": n_abstain}

```